United States Patent Application Publication	20250278620
Kind Code	A1
Publication Date	September 04, 2025
Inventor(s)	Hunter; Kevin Lee et al.

Hardware Architecture For Processing Data In Sparse Neural Network

Abstract

A hardware accelerator that is efficient at performing computations related to a sparse neural network. The sparse neural network may be associated with a plurality of nodes. One of the nodes includes one or more sparse tensors. The accelerator may compress the sparse tensor to a dense tensor. The sparse tensor may also be structured so that the dense locations in the tensor are blocked or partitioned. The accelerator may transpose the weight tensor and align the partitions of the tensor with the hardware architecture. The structured tensor has a balanced number of active values so that the active values can be processed by an efficient number of operating cycles of the accelerator. The accelerator may also perform bitwise and operation to determine the location of dense pairs in two sparse tensors to reduce the number of computations.

Inventors: **Hunter; Kevin Lee (Sunnyvale, CA), Ahmad; Subutai (Palo Alto, CA)**

Applicant: **Numenta, Inc.** (Redwood City, CA)

Family ID: **80931423**

Appl. No.: **19/211183**

Filed: **May 18, 2025**

Related U.S. Application Data

parent US continuation 17332181 20210527 PENDING child US 19211183
us-provisional-application US 63087641 20201005

Publication Classification

Int. Cl.: **G06N3/063** (20230101); **G06N3/048** (20230101); **G06N3/08** (20230101)

U.S. Cl.:

CPC **G06N3/063** (20130101); **G06N3/048** (20230101); **G06N3/08** (20130101);

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATIONS [0001] The is a continuation of U.S. patent application Ser. No. 17/332,181, filed on May 27, 2021, which claims the benefit of U.S. Provisional Patent Application 63/087,641, filed on Oct. 5, 2020, which are incorporated by reference herein in their entirety.

FIELD OF THE DISCLOSURE
[0002] The present disclosure relates to learning and processing neural network, and more specifically to hardware architecture that is efficient at performing operations related to sparse neural networks.
BACKGROUND
[0003] The use of artificial neural networks (ANN), or simply neural networks, includes a vast array of technologies. An ANN's complexity, in terms of the number of parameters, is growing exponentially at a faster rate than hardware performance. In many cases, an ANN may have a large number of parameters. Training and inference on these networks are bottlenecked by massive linear tensor operations, multiplication and convolution. Consequently, a large amount of time and/or resource may be used for both ANN creation (e.g., training) and execution (e.g., inference).
[0004] Computing systems that execute ANNs often involve extensive computing operations including multiplication and accumulation. For example, CNN is a class of machine learning techniques that primarily uses convolution between input data and kernel data, which can be decomposed into multiplication and accumulation operations. Using a central processing unit (CPU) and its main memory to instantiate and execute machine learning systems or models of various configurations is relatively easy because such systems or models can be instantiated with mere updates to code. However, relying solely on the CPU for various operations of these machine learning systems or models would consume significant bandwidth of a central processing unit (CPU) as well as increase the overall power consumption.
SUMMARY
[0005] Embodiments relate to an artificial intelligence (AI) accelerator for performing operations related to a sparse neural network. The AI accelerator may include a memory circuit that stores a sparse weight tensor and an activation tensor corresponding to a node of the sparse neural network. The AI accelerator may also include a sparsity processing circuit that is coupled to the memory circuit. The sparsity processing circuit may determine at least a location of an active value in the sparse weight tensor. The AI accelerator may further include a multiply circuit coupled to the sparsity processing circuit. The multiply circuit may receive the active value fetched based on the location determined by the sparsity processing circuit and perform a linear operation between the active value of the sparse weight tensor and one or more values of the activation tensor.
[0006] In one embodiment, the sparse weight tensor is associated with a structure that defines a distribution of a plurality of active values in the sparse weight tensor.
[0007] In one embodiment, the structure is a block structure in which the sparse weight tensor is divided into a plurality of blocks. Each block may include a plurality of values. Each block is either active or inactive. In an active block, at least one of the values is active. In an inactive block, all of the values are inactive. The structure may be a partitioned structure in which the sparse weight tensor is divided into a plurality of partitions where each partition comprises a fixed number of active values.
[0008] In one embodiment, the sparsity processing circuit includes a number of lanes where each lane is connected to a set of multiply circuits. The number of the partitions in the sparse weight

tensor is equal to the number of lanes. The sparsity processing circuit may transpose the sparse weight tensor to align the partitions with the lanes.
[0009] In one embodiment, the activation tensor is partitioned, and the sparse weight tensor is partitioned in a first dimension and the activation tensor is partitioned in a second dimension.
[0010] In one embodiment, the sparsity processing circuit may compress the sparse weight tensor by removing inactive values in the sparse weight tensor.
[0011] In one embodiment, the sparse weight tensor has 50% or less active values. In one embodiment, the activation tensor is also sparse.
[0012] In one embodiment, the artificial intelligence accelerator may also include an activation function circuit that applies a K-winner activation function to a result of the linear operation. The K-winner activation function turns one or more values in the result to zeros to generate a sparse output. The memory circuit may further store the sparse output as a sparse activation tensor for the next node of the sparse neural network.
[0013] In one embodiment, the sparse neural network is a convolutional neural network and the linear operation is convolution.
[0014] The features and advantages described in the specification are not all inclusive and, in particular, many additional features and advantages will be apparent to one of ordinary skill in the art in view of the drawings and specification. Moreover, it should be noted that the language used in the specification has been principally selected for readability and instructional purposes, and may not have been selected to delineate or circumscribe the inventive subject matter.

Description

BRIEF DESCRIPTION OF THE DRAWINGS
[0015] The teachings of the embodiments of the present invention can be readily understood by considering the following detailed description in conjunction with the accompanying drawings.
[0016] Figure (FIG. **1** is a block diagram of a computing device, according to some embodiments.
[0017] FIG. **2**A is a conceptual diagram illustrating an example architecture of a neural network, according to an embodiment.
[0018] FIG. **2**B is a block diagram illustrating an example general operation of a node in a neural network, according to an embodiment.
[0019] FIG. **2**C through **2**E illustrates the concept of sparsity in a neural network, according to an embodiment.
[0020] FIG. **3** is a block diagram illustrating circuitry and hardware architecture of an example AI accelerator, according to an embodiment.
[0021] FIG. **4**A is a conceptual diagram illustrating various examples of sparse tensors, according to an embodiment.
[0022] FIG. **4**B illustrates several examples of pairings of sparse tensors, according to an embodiment.
[0023] FIG. **5** is a flowchart depicting an example process for performing operations related to a sparse neural network, according to an embodiment.
[0024] FIG. **6** is a block diagram illustrating an example circuitry of a computation core, according to an embodiment.
[0025] FIG. **7** a block diagram illustrating an example circuitry of a computation core with multiplexers, according to an embodiment.
[0026] FIG. **8** is a flowchart depicting an example process for performing operations related to a sparse-sparse node in a neural network, according to an embodiment.
[0027] FIG. **9**A is a block diagram illustrating the generation of a sparse operand bit vector, according to an embodiment.

[0028] FIG. **9**B is a conceptual diagram illustrating the generation of a sparse operand bit vector, according to an embodiment.

DETAILED DESCRIPTION OF EMBODIMENTS

[0029] In the following description of embodiments, numerous specific details are set forth in order to provide more thorough understanding. However, note that the present invention may be practiced without one or more of these specific details. In other instances, well-known features have not been described in detail to avoid unnecessarily complicating the description.

[0030] A preferred embodiment is now described with reference to the figures where like reference numbers indicate identical or functionally similar elements. Also in the figures, the left-most digit of each reference number corresponds to the figure in which the reference number is first used.

[0031] Reference in the specification to “one embodiment” or to “an embodiment” means that a particular feature, structure, or characteristic described in connection with the embodiments is included in at least one embodiment. The appearances of the phrase “in one embodiment” in various places in the specification are not necessarily all referring to the same embodiment.

[0032] Some portions of the detailed description that follows are presented in terms of algorithms and symbolic representations of operations on data bits within a computer memory. These algorithmic descriptions and representations are the means used by those skilled in the data processing arts to most effectively convey the substance of their work to others skilled in the art. An algorithm is here, and generally, conceived to be a self-consistent sequence of steps (instructions) leading to the desired result. The steps are those requiring physical manipulations of physical quantities. Usually, though not necessarily, these quantities take the form of electrical, magnetic or optical signals capable of being stored, transferred, combined, compared and otherwise manipulated. It is convenient at times, principally for reasons of common usage, to refer to these signals as bits, values, elements, symbols, characters, terms, numbers, or the like. Furthermore, it is also convenient at times, to refer to certain arrangements of steps requiring physical manipulations of physical quantities as modules or code devices, without loss of generality.

[0033] However, all of these and similar terms are to be associated with the appropriate physical quantities and are merely convenient labels applied to these quantities. Unless specifically stated otherwise as apparent from the following discussion, it is appreciated that throughout the description, discussions utilizing terms such as “processing” or “computing” or “calculating” or “determining” or “displaying” or “determining” or the like, refer to the action and processes of a computer system, or similar electronic computing device, that manipulates and transforms data represented as physical (electronic) quantities within the computer system memories or registers or other such information storage, transmission or display devices.

[0034] Certain aspects of the embodiments include process steps and instructions described herein in the form of an algorithm. It should be noted that the process steps and instructions of the embodiments could be embodied in software, firmware or hardware, and when embodied in software, could be downloaded to reside on and be operated from different platforms used by a variety of operating systems.

[0035] Embodiments also relate to an apparatus for performing the operations herein. This apparatus may be specially constructed for the required purposes, or it may comprise a general-purpose computer selectively activated or reconfigured by a computer program stored in the computer. Such a computer program may be stored in a computer readable storage medium, such as, but is not limited to, any type of disk including floppy disks, optical disks, CD-ROMs, magnetic-optical disks, read-only memories (ROMs), random access memories (RAMs), EPROMs, EEPROMs, magnetic or optical cards, application specific integrated circuits (ASICs), or any type of media suitable for storing electronic instructions, and each coupled to a computer system bus. A computer readable medium is a non-transitory medium that does not include propagation signals and transient waves. Furthermore, the computers referred to in the specification may include a single processor or may be architectures employing multiple processor designs for increased

computing capability.
[0036] The algorithms and displays presented herein are not inherently related to any particular computer or other apparatus. Various general-purpose systems may also be used with programs in accordance with the teachings herein, or it may prove convenient to construct more specialized apparatus to perform the required method steps. The required structure for a variety of these systems will appear from the description below. In addition, embodiments are not described with reference to any particular programming language. It will be appreciated that a variety of programming languages may be used to implement the teachings as described herein, and any references below to specific languages are provided for disclosure of enablement and best mode of the embodiments.
[0037] In addition, the language used in the specification has been principally selected for readability and instructional purposes, and may not have been selected to delineate or circumscribe the inventive subject matter. Accordingly, the disclosure set forth herein is intended to be illustrative, but not limiting, of the scope, which is set forth in the claims.
[0038] Embodiments relate to architecture of an artificial intelligence (AI) accelerator that is efficient at processing sparse nodes of a neural network. A sparse node may include a sparse tensor that has a low density of active values. In using a generic processor, the computation operation of a tensor, sparse or dense, may include computing the value in the tensor one by one. However, in a sparse tensor, since many values in the tensor are inactive (e.g., zeros) and computation with such inactive values can be skipped, the AI accelerator may determine the locations of active values in the tensor and perform computation efficiently so that the number of operations to process the tensor is reduced. The sparse tensor may also have a structure that limits the distribution patterns of the active values to further accelerate the operation of the AI accelerator. Here, an active value may be referred to a value that requires computation, such as a non-zero value. An inactive value may be referred to a value whose computation may be skipped. For example, in a dot product computation, an inactive value may be zero and the computation associated with the zero may be skipped because zero does not contribute to the final result value of the dot product.
Example Computing Device Architecture
[0039] FIG. **1** is a block diagram of an example computing device **100** for processing one or more sparse neural networks, according to an embodiment. A computing device **100** may be a server computer, a personal computer, a portable electronic device, a wearable electronic device (e.g., a smartwatch), an IoT device (e.g., a sensor), smart/connected appliance (e.g., a refrigerator), dongle, a device in edge computing, a device with limited processing power, etc. The computing device **100** may include, among other components, a central processing unit (CPU) **102**, an artificial intelligence accelerator (AI accelerator) **104**, a graphical processing unit (GPU) **106**, system memory **108**, a storage unit **110**, an input interface **114**, an output interface **116**, a network interface **118**, and a bus **120** connecting these components. In various embodiments, computing device **100** may include additional, fewer or different components.
[0040] While some of the components in this disclosure may at times be described in a singular form while other components may be described in a plural form, various components described in any system may include one or more copies of the components. For example, a computing device **100** may include more than one processor such as CPU **102**, AI accelerator **104**, and GPU **106**, but the disclosure may refer the processors to as “a processor” or “the processor.” Also, a processor may include multiple cores.
[0041] CPU **102** may be a general-purpose processor using any appropriate architecture. CPU **102** retrieves and executes computer code that includes instructions, when executed, that may cause CPU **102** or another processor, individually or in combination, to perform certain actions or processes that are described in this disclosure. Instructions can be any directions, commands, or orders that may be stored in different forms, such as equipment-readable instructions, programming instructions including source code, and other communication signals and orders. Instructions may

be used in a general sense and are not limited to machine-readable codes. CPU **102** may be used to compile the instructions and also determine which processors may be used to performed certain tasks based on the commands in the instructions. For example, certain machine learning computations may be more efficient to be processed using AI accelerator **104** while other parallel computations may be better to be processed using GPU **106**.

[0042] AI accelerator **104** may be a processor that is efficient at performing certain machine learning operations such as tensor multiplications, convolutions, tensor dot products, etc. In various embodiments, AI accelerator **104** may have different hardware architectures. For example, in one embodiment, AI accelerator **104** may take the form of field-programmable gate arrays (FPGAs). In another embodiment, AI accelerator **104** may take the form of application-specific integrated circuits (ASICs), which may include circuits along or circuits in combination with firmware.

[0043] GPU **106** may be a processor that includes highly parallel structures that are more efficient than CPU **102** at processing large blocks of data in parallel. GPU **106** may be used to process graphical data and accelerate certain graphical operations. In some cases, owing to its parallel nature, GPU **106** may also be used to process a large number of machine learning operations in parallel. GPU **106** is often efficient at performing the same type of workload many times in rapid succession.

[0044] While, in FIG. **1**, the processors CPU **102**, AI accelerator **104**, and GPU **106** are illustrated as separated components, in various embodiments the structure of one processor may be embedded in another processor. For example, one or more examples of the circuitry of AI accelerator **104** disclosed in different figures of this disclosure may be embedded in a CPU **102**. The processors may also be included in a single chip such as in a system-on-a-chip (SoC) implementation. In various embodiments, computing device **100** may also include additional processors for various specific purposes. In this disclosure, the various processors may be collectively referred to as "processors" or "a processor."

[0045] System memory **108** includes circuitry for storing instructions for execution by a processor and for storing data processed by the processor. System memory **180** may take the form of any type of memory structure including, for example, dynamic random access memory (DRAM), synchronous DRAM (SDRAM), double data rate (DDR, DDR2, DDR3, etc.) RAMBUS DRAM (RDRAM), static RAM (SRAM) or a combination thereof. System memory **108** usually takes the form of volatile memory.

[0046] Storage unit **110** may be a persistent storage for storing data and software applications in a non-volatile manner. Storage unit **110** may take the form of read-only memory (ROM), hard drive, flash memory, or another type of non-volatile memory device. Storage unit **110** stores the operating system of the computing device **100**, various software applications **130** and machine learning models **140**. Storage unit **110** may store computer code that includes instructions that, when executed, cause a processor to perform one or more processes described in this disclosure.

[0047] Applications **130** may be any suitable software applications that operate at the computing device **100**. An application **130** may be in communication with other devices via network interface **118**. Applications **130** may be of different types. In one case, an application **130** may be a web application, such as an application that runs on JavaScript. In another case, an application **130** may be a mobile application. For example, the mobile application may run on Swift for iOS and other APPLE operating systems or on Java or another suitable language for ANDROID systems. In yet another case, an application **130** may be a software program that operates on a desktop operating system such as LINUX, MICROSOFT WINDOWS, MAC OS, or CHROME OS. In yet another case, an application **130** may be a built-in application in an IoT device. An application **130** may include a graphical user interface (GUI) that visually renders data and information. An application **130** may include tools for training machine leaning models **140** and/or perform inference using the trained machine learning models **140**.

[0048] Machine learning models **140** may include different types of algorithms for making

inferences based on the training of the models. Examples of machine learning models **140** include regression models, random forest models, support vector machines (SVMs) such as kernel SVMs, and artificial neural networks (ANNs) such as convolutional network networks (CNNs), recurrent network networks (RNNs), autoencoders, long short term memory (LSTM), reinforcement learning (RL) models. Some of the machine learning models may include a sparse network structure whose detail will be further discussed with reference to FIG. **2**B through **2**D. A machine learning model **140** may be an independent model that is run by a processor. A machine learning model **140** may also be part of a software application **130**. Machine learning models **140** may perform various tasks.

[0049] By way of example, a machine learning model **140** may receive sensed inputs representing images, videos, audio signals, sensor signals, data related to network traffic, financial transaction data, communication signals (e.g., emails, text messages and instant messages), documents, insurance records, biometric information, parameters for manufacturing process (e.g., semiconductor fabrication parameters), inventory patterns, energy or power usage patterns, data representing genes, results of scientific experiments or parameters associated with the operation of a machine (e.g., vehicle operation) and medical treatment data. The machine learning model **140** may process such inputs and produce an output representing, among others, identification of objects shown in an image, identification of recognized gestures, classification of digital images as pornographic or non-pornographic, identification of email messages as unsolicited bulk email ('spam') or legitimate email ('non-spam'), prediction of a trend in financial market, prediction of failures in a large-scale power system, identification of a speaker in an audio recording, classification of loan applicants as good or bad credit risks, identification of network traffic as malicious or benign, identity of a person appearing in the image, processed natural language processing, weather forecast results, patterns of a person's behavior, control signals for machines (e.g., automatic vehicle navigation), gene expression and protein interactions, analytic information on access to resources on a network, parameters for optimizing a manufacturing process, predicted inventory, predicted energy usage in a building or facility, web analytics (e.g., predicting which link or advertisement that users are likely to click), identification of anomalous patterns in insurance records, prediction on results of experiments, indication of illness that a person is likely to experience, selection of contents that may be of interest to a user, indication on prediction of a person's behavior (e.g., ticket purchase, no-show behavior), prediction on election, prediction/detection of adverse events, a string of texts in the image, indication representing topic in text, and a summary of text or prediction on reaction to medical treatments. The underlying representation (e.g., photo, audio and etc.) can be stored in system memory **108** and/or storage unit **110**.

[0050] Input interface **114** receives data from external sources such as sensor data or action information. Output interface **116** is a component for providing the result of computations in various forms (e.g., image or audio signals). Computing device **100** may include various types of input or output interfaces, such as displays, keyboards, cameras, microphones, speakers, antennas, fingerprint sensors, touch sensors, and other measurement sensors. Some input interface **114** may directly work with a machine learning model **140** to perform various functions. For example, a sensor may use a machine learning model **140** to infer interpretations of measurements. Output interface **116** may be in communication with humans, robotic agents or other computing devices.

[0051] The network interface **118** enables the computing device **100** to communicate with other computing devices via a network. The networks may include, but are not limited to, Local Area Networks (LANs) (e.g., an Ethernet or corporate network) and Wide Area Networks (WANs). When multiple nodes or components of a single node of a machine learning model **140** is embodied in multiple computing devices, information associated with various processes in the machine learning model **140**, such as temporal sequencing, spatial pooling and management of nodes may be communicated between computing devices via the network interface **118**.

Example Neural Network Architecture

[0052] FIG. **2**A is a conceptual diagram illustrating an example architecture of a neural network **200**, according to an embodiment. The illustrated neural network **200** shows a generic structure of a neural network. Neural network **200** may represent different types of neural networks, including convolutional network networks (CNNs), recurrent network networks (RNNs), autoencoders, and long short term memory (LSTM). In various embodiments, customized changes may be made to this general structure. Neural network **200** may also be a hierarchical temporal memory system as described, for example, in U.S. Patent Application Publication No. 2020/0097857, published on May 26, 2020, which is incorporated hereto by reference in its entirety.

[0053] Neural network **200** includes an input layer **202**, an output layer **204** and one or more hidden layers **206**. Input layer **202** is the first layer of neural network **200**. Input layer **202** receives input data, such as image data, speech data, text, etc. Output layer **204** is the last layer of neural network **200**. Output layer **204** may generate one or more inferences in the form of classifications or probabilities. Neural network **200** may include any number of hidden layers **206**. Hidden layer **200** are intermediate layers in neural network **200** that perform various operations. Neural network **200** may include additional or fewer layers than the example shown in FIG. **2**A. Each layer may include one or more nodes **210**. The number of nodes in each layer in the neural network **200** shown in FIG. **2**A is an example only. A node **210** may be associated with certain weights and activation functions. In various embodiments, the nodes **210** in neural network **200** may be fully connected or partially connected.

[0054] Each node **210** in neural network **200** may be associated with different operations. For example, in a simple form, neural network **200** may be a vanilla neural network whose nodes are each associated with a set of linear weight coefficients and an activation function. In another embodiment, neural network **200** may be an example convolutional neural network (CNN). In this example CNN, nodes **210** in one layer may be associated with convolution operations with kernels as weights that are adjustable in the training process. Nodes **210** in another layer may be associated with spatial pooling operations. In yet another embodiment, neural network **200** may be a recurrent neural network (RNN) whose nodes may be associated with more complicated structures such as loops and gates. In a neural network **200**, each node may represent a different structure and have different weight values and a different activation function.

[0055] FIG. **2**B is a block diagram illustrating an example general operation of a node **210** in neural network **200**, according to an embodiment. A node **210** may receive an input activation tensor **220**, which can be an N-dimensional tensor, where N can be greater than or equal to one. Input activation tensor **220** may be the input data of neural network **200** if node **210** is in the input layer **202**. Input activation tensor **220** may also be the output of another node in the preceding layer. Node **210** may apply a weight tensor **222** to input activation tensor **220** in a linear operation **224**, such as addition, scaling, biasing, tensor multiplication, and convolution in the case of a CNN. The result of linear operation **224** may be processed by a non-linear activation **226** such as a step function, a sigmoid function, a hyperbolic tangent function (tanh), and rectified linear unit functions (ReLU). The result of the activation is an output activation tensor **228** that is sent to a subsequent connected node that is in the next layer of neural network **200**. The subsequent node uses output activation tensor **228** as the input activation tensor **220**.

[0056] In various embodiments, a wide variety of machine learning techniques may be used in training neural network **200**. Neural network **200** may be associated with an objective function (also commonly referred to as a loss function), which generates a metric value that describes the objective goal of the training process. The training may intend to reduce the error rate of the model in generating predictions. In such a case, the objective function may monitor the error rate of neural network **200**. For example, in object recognition (e.g., object detection and classification), the objective function of neural network **200** may be the training error rate in classifying objects in a training set. Other forms of objective functions may also be used. In various embodiments, the

error rate may be measured as cross-entropy loss, L1 loss (e.g., the sum of absolute differences between the predicted values and the actual value), L2 loss (e.g., the sum of squared distances) or their combinations.

[0057] The weights and coefficients in activation functions of neural network may be adjusted by training and also be constrained by sparsity and structural requirements. Sparsity will be further discussed with reference to FIG. **2**C through **2**E and structural requirements will be further discussed with reference to FIG. **4**A. Training of neural network **200** may include forward propagation and backpropagation. In forward propagation, neural network **200** performs the computation in the forward direction based on outputs of a preceding layer. The operation of a node **210** may be defined by one or more functions, such as linear operation **224** and non-linear activation **226**. The functions that define the operation of a node **210** may include various computation operations such as convolution of data with one or more kernels, pooling, recurrent loop in RNN, various gates in LSTM, etc. The functions may also include an activation function that adjusts the output of the node.

[0058] Each of the functions in neural network **200** may be associated with different coefficients (e.g., weights and kernel coefficients) that are adjustable during training. After an input is provided to neural network **200** and passes through neural network **200** in the forward direction, the results may be compared to the training labels or other values in the training set to determine the neural network's performance. The process of prediction may be repeated for other samples in the training sets to compute the overall value of the objective function in a particular training round. In turn, neural network **200** performs backpropagation by using gradient descent such as stochastic gradient descent (SGD) to adjust the coefficients in various functions to improve the value of the objective function.

[0059] Multiple rounds of forward propagation and backpropagation may be performed. Training may be completed when the objective function has become sufficiently stable (e.g., neural network **200** has converged) or after a predetermined number of rounds for a particular set of training samples. The trained neural network **200** can be used for making inferences or another suitable task for which the model is trained.

[0060] FIG. **2**C through **2**E illustrates the concept of sparsity in a neural network **200**, according to various embodiments. Each of FIGS. **2**C, **2**D, and **2**E shows the operation within a node **210** with different degrees of sparsity and is a graphical illustration of the flowchart shown in FIG. **2**B. A circle in FIGS. **2**C, **2**D, and **2**E represents a value in a tensor. In a neural network **200** with L hidden layers, the notation y.sup.l denotes output activation tensor **228** from layer l and y.sup.l-1 denotes the output activation tensor **228** in the preceding layer l-1 or the input activation tensor **220** of layer l. W.sup.l and u.sup.l represent respectively the weight tensor **222** and biases for each node. In a neural network node **210** that has a dense weight tensor W′, the feed-forward outputs are calculated as follow:

[00001] $\hat{y}^l = W^l$ .Math. $y^{l-1} + u^l$ Equation1 $y^l = f(\hat{y}^l)$ Equation2

where f is any activation function, such as tanh or ReLU and ŷ.sup.l is the output of the linear operation before an activation function is applied.

[0061] The above relationship may be conceptually represented as a block diagram as illustrated in FIG. **2**B. Graphically, a dense node with dense weights and a dense activation function such as tanh or ReLU is illustrated in FIG. **2**C. In FIG. **2**C, the result ŷ.sup.l after the linear operation is dense with most of the values being non-zero. The active values (e.g., non-zero values) are represented by the shaded circles. The activation function also results in a dense output y.sup.l in which a majority of the values are still active, which are also represented by the shaded circles.

[0062] Here, a value being active may refer to a value whose mathematical operation will need to be included in order to perform the overall computation. For example, in the context of matrix multiplication, convolution, or dot product, an active value may be a non-zero value because the

mathematical operation, such as addition and multiplication, of the non-zero value will need to be included in order to get to the correct result of the matrix multiplication, convolution, or dot product. A value being inactive may refer to a value whose mathematical operation may be skipped. For example, in the context of matrix multiplication, convolution, or dot product, an inactive value is zero because the mathematical operation involving zero, such as addition and multiplication, may be skipped without affecting the final result. A weight tensor is dense if the percentage of active values in the tensor exceeds a threshold. Likewise, an activation is dense if the activation function will result in a number of output values in the output activation tensor y.sup.l being dense and the percentage of the active values exceeding a threshold. Using ReLU as an example, ReLU sets values that are lower than a level (e.g., 0) as 0 and allows values that are greater than the level to retain the values. Hence, it is expected that ReLU will generate about half active values if the values in the intermediate tensor ŷ.sup.l are roughly equally distributed around the level. A tensor output that has about half of the values being non-zero is often considered as dense. In FIG. **2**C, since the weight tensor is dense and the activation layer will also generate a dense value, the node **240** can be considered as a weight-dense and activation-dense node **240**, or simply referred to as dense-dense node **240**.

[0063] The degree of sparsity for a tensor to be considered sparse may vary, depending on embodiments. In one embodiment, the number of active values in a tensor is fewer than 50% to be considered a sparse tensor. In one embodiment, the number of active values in a tensor is fewer than 40% to be considered a sparse tensor. In one embodiment, the number of active values in a tensor is fewer than 30% to be considered a sparse tensor. In one embodiment, the number of active values in a tensor is fewer than 20% to be considered a sparse tensor. The number of active values in a tensor is fewer than 15% to be considered a sparse tensor. The number of active values in a tensor is fewer than 10% to be considered a sparse tensor. The number of active values in a tensor is fewer than 5% to be considered a sparse tensor. The number of active values in a tensor is fewer than 4% to be considered a sparse tensor. The number of active values in a tensor is fewer than 3% to be considered a sparse tensor. The number of active values in a tensor is fewer than 3% to be considered a sparse tensor. The number of active values in a tensor is fewer than 2% to be considered a sparse tensor. The number of active values in a tensor is fewer than 1% to be considered a sparse tensor. The number of active values in a tensor is fewer than 0.8% to be considered a sparse tensor. The number of active values in a tensor is fewer than 0.5% to be considered a sparse tensor. The number of active values in a tensor is fewer than 0.2% to be considered a sparse tensor. The number of active values in a tensor is fewer than 0.1% to be considered a sparse tensor. The number of active values in a tensor is fewer than 0.01% to be considered a sparse tensor.

[0064] FIG. **2**D is a conceptual diagram that illustrates a sparse-dense node **250**, according to an embodiment. Compared to the node **240** in FIG. **2**C, node **250** has sparse weights that are illustrated by having much fewer connected lines. Despite being illustrated as a dense tensor, the input y.sup.l-1 can be a dense tensor or a sparse tensor, depending on the previous node's sparsity. The weights of this node **240** are sparse, meaning there are a large number of inactive values (e.g., zeros) in the weight tensor. A sparse weight tensor may be achieved by imposing a constraint on node **240** to limit the maximum number of active values in the weight tensor. After the linear operation, the intermediate tensor ŷ.sup.l is likely to be dense because the linear operation, such as tensor multiplication or convolution, likely spreads the number of active values in the tensor. After the linear operation, the non-linear activation **226** step is the same as the node **240** in FIG. **2**C. For example, the ReLU activation function will get around half of the values as zeros. Overall, the output tensor y.sup.l is still a dense tensor since about half of the values are dense. In this example, node **250** may be referred to as a weight-sparse and activation-dense node or simply sparse-dense node.

[0065] FIG. **2**E is a conceptual diagram that illustrates a sparse-sparse node **260**, according to an

embodiment. Compared to node **250** in FIG. **2**D, node **260** also has sparse weights, but it also a sparse activation function that generates a sparse output. The input y.sup.l-1 can be a dense tensor or a sparse tensor, depending on the previous node's sparsity. In this example, the input y.sup.l-1 is illustrated as a sparse tensor. Even though the weights of this node **260** are sparse, after the linear operation, the intermediate tensor ŷ.sup.l is likely to be dense because the linear operation likely spreads the number of non-zero values in the tensor. After the linear operation, a sparse activation function called K-winner activation is used instead of a dense activation such as ReLU activation function. K-winner activation selects the top K values in the intermediate tensor ŷ.sup.l and force all other values, non-zero or not, to zeros. K may be a constraint set to maintain the sparsity of node **260** and may be set as a percentage of the total number of values in a tensor. For example, K may be 30%, 20%, 15%, 10%, 5%, etc., depending on the selection. The output tensor y.sup.l is a sparse tensor after the K-winner activation function that restraints the number of active values in the tensor. In this example, node **260** may be referred to as a weight-sparse and activation-sparse node or simply sparse-sparse node.

[0066] Neural network **200** with one or more nodes that have the sparse-dense or sparse-sparse structure may be referred to as a sparse neural network. A sparse neural network may be a hierarchical temporal memory system. In various embodiments, while a sparse neural network may include a large number of sparse nodes, the sparse neural network may also include some dense nodes. Also, a sparse node may be a sparse-sparse node **260** or a sparse-dense node **250**.

[0067] A sparse neural network often has improved performance in terms of speed in training and inference because the large number of inactive values in the network allows the network to skip many mathematical operations. For example, many common operations in neural networks, such as convolution and tensor multiplication, may be converted to dot products. Oftentimes a processor uses dot products to compute those operations in neural networks. Zeros in the tensors will significantly simplify the number of multiplications and additions needed to perform in a dot product. In many cases, sparse neural networks may model the structure of a human brain, which appears to also rely on a large degree of sparsity. Those sparse neural networks often not only have improved speed compared to dense neural networks but also increase inference accuracy particularly in the cases of noisy environments. For example, sparse neural networks reduce the number of parameters necessary to achieve an equivalent result accuracy, leading to savings in computational infrastructure, execution time, latency, power and therefore costs. They also exhibit increased robustness to noise in real-world situations. In Edge and IoT applications, a sparse network may fit on a limited deployment platform where an equivalent dense network would not.

Example Circuitry for AI Accelerator

[0068] FIG. **3** is a block diagram illustrating circuitry and hardware architecture of an example AI accelerator **300**, according to an embodiment. AI accelerator **300** may be a circuit that is efficient at performing operations related to a sparse neural network. AI accelerator **300** may be an example of AI accelerator **104** or may also be embedded as part of a larger processor, such as CPU **102**. In various embodiments, AI accelerator **300** may include fewer or additional components than the example shown in FIG. **3**. For example, in one embodiment, AI accelerator **300** shown in FIG. **3** only illustrates blocks that are relevant to computations related to accelerating the operation of a sparse neural network and other components may not be shown. In one embodiment, AI accelerator **300** includes internal memory **310** and one or more computation cores **320** that perform computations in parallel.

[0069] Internal memory **310** may be the dedicated memory for AI accelerator **300** that is used for storage of data fetched from system memory **106** and data outputted by computation cores **320**. The data stored in internal memory **310** may include input data of neural network **200**, weights and other coefficients in neural network **200**, intermediate data of neural network **200**, such as output activation tensor **228** that is outputted by each node **210**, loss function coefficients, and other suitable data that are related to the operation of neural network **200**. For each node **210**, input

activation tensor **220** may be saved in internal memory **310**. The input activation tensor **220** may be divided into multiple units and are sent to various computation cores **320** to process in parallel. The outputs of computation cores **320***s* may be recombined as output activation tensor **228**, which is an output of a node **210**. After the operations of the nodes **210** in a layer of neural network **200** are completed, operations of nodes **210** in the next layer may begin. The output activation tensor **228** is then fetched again to one or more computation core **320** as the input activation tensor **220** of a succeeding node **210** in the next layer. The process repeats until the operations reach the output layer **204**. In some embodiments, the data stored in internal memory **310** may be sparse tensors that include zeros in various locations. In some embodiments, some data in internal memory **310** may also be compressed to dense tensors by removing zeros in the tensors. Compression of sparse tensors will be discussed in further detail.

[0070] In some embodiments, an AI accelerator **300** may not need to include internal memory **310**. Instead, data are directly fetched and written to the system memory **106**.

[0071] A computation core **320** is a circuit that includes a number of multiply circuits **330** that perform tensor operations such as the multiplications part of dot products, tensor multiplications, convolutions. Common machine learning operations such as tensor multiplications and convolutions may be converted to dot products and be performed by multiply circuits **330**. A computation core **320** may include a number of multiply circuits for performing computations in parallel.

[0072] A multiply circuit **330** may take various forms. In one embodiment, a multiply circuit **330** is a multiply-accumulate circuit (MAC) that includes multiply units and accumulators. The multiply units may be used to perform multiplications and additions. A multiply unit is a circuit with a known structure and may be used for binary multiplication or floating-point multiplication. An accumulator is a memory circuit that receives and stores values from the multiply units. The values may be stored individually or added together in the accumulator.

[0073] Computation core **320** may include circuitry upstream of multiply circuits **330** for pre-processing of various tensors such as by dividing an input activation tensor into smaller units and by compressing and converting sparse tensors to a form that is efficient for the multiply circuits **330** to process. An activation buffer **352** is a buffer circuit and related data-processing circuit for performing data processing of an input activation tensor **220** for a node **210**. For example, normally an input activation tensor **220** may have a size that is significantly larger than the capacity of a multiply circuit **330**. The input activation tensor **220** may be divided into multiple data subunits and be processed in parallel by different multiply circuits **330**. Activation buffer **352** may include circuitry that divides the input activation tensor **220** or include different addresses for various multiply circuits **330** to fetch different portions of the input activation tensor **220**. In some embodiments, activation buffer **352** may fetch the tensor values from internal memory **310**. In some cases, only the active values are fetched to activation buffer **352**.

[0074] Activation buffer **352** may also perform a transpose operation of the input activation tensor **220** by fetching data values in the input activation tensor **220** in an order different from the order in internal memory **310**. In some cases, an input activation tensor **220** may be saved in internal memory **310** under certain dimensions such as X by Y by Z while the division of data subunits may be more efficient under the dimension Y by Z by X. The efficiency of storage and operation of data under certain dimensions may depend on the hardware landscape such as the multiplier arrangement in a multiply circuit **330** and memory structure. Some examples of transpose operations that are efficient for sparse tensors will be discussed in detail with reference to FIGS. **6** and **7**.

[0075] A weight buffer **350** and sparsity processing circuit **354** are other examples of circuitry upstream of multiply circuits **330** for pre-processing of various tensors. For an operation with respect to a given node **210** in neural network **200**, weight buffer **350** fetches the tensor values of weight tensor **222** from internal memory **310** or system memory **106**. Similar to activation buffer

352, in some cases weight buffer **350** may only fetch the active values in weight tensor **222**.
[0076] Sparsity processing circuit **354** may include different types of circuits that are used to pre-process weight tensor **222** and input activation tensor **220**. Weight tensor **222** and input activation tensor **220** may be associated with different degrees of sparsity. For example, in one case, weight tensor **222** may be sparse while input activation tensor **220** may be dense. In another case, both weight tensor **222** and input activation tensor **220** are sparse. Sparsity processing circuit **354** may pre-process weight tensor **222** and input activation tensor **220** in different ways, depending on their sparsity. For example, in some embodiments, weight tensor **222** and input activation tensor **220** may be processed separately. In some embodiments, when both weight tensor **222** and input activation tensor **220** are sparse, sparsity processing circuit **354** may process the two tensors together. Various example structures of sparsity processing circuit **354** and sparsity processing approaches are discussed later in this disclosure in association with FIG. **6** through FIG. **9**B.
[0077] The pre-processing in sparsity processing circuit **354** may include identifying locations of active values in the weight tensor **222** and input activation tensor **220**. Sparsity processing circuit **354** may scan through a sparse tensor and identify the locations of the active values in the sparse tensor. The locations may take the form of the locations in the tensor (e.g., a location at the third row and the fifth column in the tensor) and may also take the form of memory addresses of active values (e.g., an active value being saved in the memory address of 0xC0010000). Sparsity processing circuit **354** may only transmit the active values to multiply circuits **330** for computations. In some embodiments, sparsity processing circuit **354** may identify dense pairs that have active values at the same tensor location in both weight tensor **222** and input activation tensor **220**. Sparsity processing circuit **354** may only transmit the dense pairs to multiply circuits **330** for computations. In other words, in some cases, sparsity processing circuit **354** may exclude the transmission of inactive values in weight tensor **222** or input activation tensor **220** to multiply circuits **330**.
[0078] The pre-processing in sparsity processing circuit **354** may also include compressing a sparse tensor to a dense tensor. In various computations such as dot products and other multiplications, the results will be zero if one of the input values is zero. As such, the processing of those inactive values may be skipped in the multiply circuits **330**. In some cases, when two tensors are multiplied, only multiplications of two active values are to be computed. As such, in some embodiments, sparsity processing circuit **354** may compress a sparse tensor by converting the sparse tensor into a dense tensor. The number of multiplication operations to be performed by multiply circuits **330** may be significantly reduced after inactive values are removed from the tensors. By way of example, if a dot product is performed between two sparse tensors that each has about 10% of active values, it is expected to only 1% of the multiplication operations will need to be performed. The rest of the positions are either the multiplications of two zeros or multiplications of a non-zero value and a zero. By removing the inactive value (e.g., zeros) in the tensors, sparsity processing circuit **354** may speed up the computations for multiply circuits **330**. In some embodiments, the tensors fetched to sparsity processing circuit **354** may also be structured so that sparsity processing circuit **354** can remove the zeros in those tensors more efficiently. The structure of tensors will be further discussed in FIG. **4**A. In some embodiments, sparsity processing circuit **354** may also store the addresses of active values in the tensors so that the compressed tensors and output tensors generated by multiply circuits **330** may be expanded back to sparse tensors.
[0079] Sparsity processing circuit **354** may also perform other data pre-processing such as transposing weight tensor **222** and input activation tensor **220**. Sparsity processing circuit **354** may also divide the tensors in a way that is efficient for multiply circuits **330** to process. The pre-processed tensors are fetched and sent to multiply circuits **330** to perform computations with input activation tensor **220**.
[0080] After results of multiply circuits **330** are computed, the results are sent to an adder tree **360** to generate an intermediate output tensor ŷ.sup.l. In some embodiments, input activation tensor **220**

is divided into multiple subunits for parallel processing in the multiply circuits **330**. The results of the multiply circuits **330** are then combined in adder tree **360**. For example, in performing a dot product, multiply circuits **330** perform the multiplication and accumulation parts of the dot product and the results of different multiply circuits **330** are added together at the adder tree **360** to generate the final result. Alternatively, the accumulation parts may be performed in the adder tree, depending on the hardware architecture and the operations. In some cases, a bias factor **364** may also be fetched from internal memory **310**. The bias factor **364** may be added to each value in the output of the adder tree **360**.

[0081] An activation function circuit **370** is a circuit downstream of adder tree **360** to perform the operation specified in the activation function. Activation function circuit **370** includes a number of comparator circuits that are used for the ReLU activation function. Activation function circuit **370** may also include comparator trees for determining top K highest values in a tensor in the case of a sparse K-winner activation function. Activation function circuit **370** generates the output activation tensor **228** from the intermediate output tensor. Activation function circuit **370** may set a number of values in the intermediate output tensor to zero, depending on the type of activation function. Hence, output activation tensor **228** may be a dense or sparse tensor. In some embodiments, one or more input tensors are previously compressed, activation function circuit **370** may also expand the output activation tensor **228** back to the original size. Output activation tensor **228** is transmitted to internal memory **310** or system memory **106** as the output of a particular node **210**. The output activation tensor **228** is fetched subsequently as input activation tensor **220** when another round of operations related to a subsequent node **210** begins.

[0082] The use of a sparse neural network and an AI accelerator that is efficient at operating with the sparse neural network reduces the number of computations and power consumptions of the AI accelerator. The sparse neural network also reduces storage requirements and working memory bandwidth. The AI accelerator improves the speed of a computing device and is suitable for use in computing devices that have limited power or computing capacity, such as IoT devices and in the case of edge computing.

Example Structured Sparse Tensor Configurations

[0083] FIG. **4**A is a conceptual diagram illustrating various examples of sparse tensors, according to an embodiment. Structured sparse tensors may be used as the data format of a sparse neural network. Such a format is efficient for an AI accelerator **300** to perform computations. For example, a properly structured tensor with active values arranged in a certain structured manner may allow sparsity processing circuit **354** to process and compress the tensor in an efficient manner, thereby further speeding up the neural network.

[0084] Tensor **402** is an example unstructured tensor. Tensor **402** and various tensors in FIG. **4**A are illustrated as 2-dimensional tensors with an x-direction (row) and a y-direction (column). In actual data, the tensors used in neural network **200** may include in any number of dimensions, from one to many. The discussions using 2-dimensional tensors may be extended to any number of dimensions. Whether a tensor is structured depends on whether the active values are distributed in any specific manner according to one or more constraints. According to an embodiment, the manners to arrange the active values may be referred to as block and partition. Each of those manners will be discussed in further detail in association with tensor **404** through tensor **438**. In FIG. **4**A, the active values (e.g., non-zero values) are represented by the shaded cells and the inactive values (e.g., zeros) are represented by the white blocks. In unstructured tensor **402**, the active values are distributed randomly. For example, in the x-direction, the first row of tensor **402** has 4 active values; the second row of tensor **402** has 4 active values; and the third row of tensor **402** has only 1 active value. The active values in unstructured tensor **402** are generated based on the training of neural network **200** without any constraints imposed on how the active values should be placed.

[0085] The use of unstructured tensors in an AI accelerator **300** may significantly slow down the speed of operation due to the sparse marshalling problem in identifying the randomly located active

values. As mentioned in FIG. **3**, to speed up a sparse neural network, a sparse tensor may be compressed to a dense tensor so that the computations related to value locations that are zero are skipped. However, in an unstructured tensor **402** that has active values occurring without an easily identifiable pattern, an AI accelerator may need to spend significant resources and time to scan through the tensor to identify the locations of active values. The time for scanning through the tensor may sometimes even be longer than performing the tensor multiplication in a brute-force manner such as by performing multiplications of two sparse tensors on all data locations of the tensors without identifying the sparse locations.

[0086] The marshalling problem may be illustrated by an example. The expected number of multiply-accumulate operations for a sparse-sparse (both tensors are sparse) dot product is the product of the tensors' densities. In a 1600-element dot product, if the first tensor's density is 5% and the second tensor's density is 12.5%, the expected number of the multiply-accumulate operations between two active values is only 10. This represents 160 times of computation reduction. To realize this computation reduction, the sparse tensors may be distilled by sparsity processing circuit **354** to eliminate the operand pairs that have an inactive value involved and keep only the mutually active operand pairs from each sparse tensor. This distillation process may be referred to as a sparse to dense compression. However, without specific structured tensors and circuitry, rendezvousing these mutually active pairs can be a challenging problem. Also, in an unstructured tensor, the positions of active values within a tensor usually do not follow an algorithmic pattern. During compression from a sparse tensor to a dense tensor, coordinates will need to be associated with the active values. There will be storage and performance overhead in an AI accelerator for accessing these coordinates. General hardware circuitry, whether conventional CPU, GPU, FPGA, or ASIC, may take a significant time to compare both tensors to determine the locations with active values in both tensors. The time or the hardware footprint needed to perform the searching may rival a dense operation that conducts the dot products in all 1600 locations by vector processing with single instruction multiple data (SIMD) units. The searching of those locations may be referred to as the marshalling problem.

[0087] According to an embodiment, the sparsity of tensors in a neural network **200** may be constrained so that the active values are spatially structured. For example, structured tensors may be achieved in the training of neural network **200** by imposing one or more constraints on how the active values are distributed. The tensors **404** through **438** illustrate two types of structure, which are referred to as block structure and partitioned structure. A tensor may also be in a combination of these two types of structures. In a block structure, a tensor may be divided into blocks, which are a group of data value locations in the tensor. In the block structure, the active values are concentrated in a subset of blocks, leaving the rest of the blocks completely inactive. In a partitioned structure, the tensor may be divided into sub-volumes. One or more constraints may be imposed equally on each sub-volume. For example, the number of active values in each sub-volume may be a fixed number so that the partitions have a balanced number of active values. The partitioned structure results in less variability of the sparsity, which in turn reduces the combinatorics of the marshalling problem. The constraints of blocks and partitions may be imposed on one or more dimensions of the tensor. A tensor may also have both the block and partitioned structures in one or more dimensions.

[0088] Tensors **404** through **438** illustrate various examples of structures in different dimensions, according to different embodiments. In tensor **404**, the tensor is divided into blocks in x-dimension. Each block includes 1×4 value locations. Each block is either active or inactive. In an active block, at least one of the values is active. In an inactive block, all of the values are inactive. In tensor **406**, the tensor is divided into partitions in x-dimension. Each row is a partition. A constraint is imposed on tensor **404** so that each partition (each partition) has the same number (4) of active values. In tensor **408**, both block structure and petitioned structure are imposed in x-dimension. Similar to tensor **404**, tensor **408** is divided into 1×4 blocks. Each row in tensor **408** has one and only one

active block, which is a condition imposed by the partition.
[0089] Tensor **412** through **438** illustrate additional structures that are in different dimensions and different combinations. For example, tensor **412** is a block structure in y-dimension. Tensor **414** is a block structure in both x and y dimensions. Each block includes 2×2 value locations. In tensor **416**, block structure is imposed in y-dimension while the partition structure is imposed in x-dimension. As such, each row (x-dimension) has four dense vertical blocks. Tensor **418** is divided by 2×2 x-y blocks. Partitioning is imposed in x-dimension so that each row in tensor **418** has 2 blocks. Tensors **422**, **424**, **426**, **428**, **432**, **434**, and **436** are additional examples of different combinations of block and partitioned structures. Tensor **438** is divided by 2×2 x-y blocks. Partitioning is imposed in both x-dimension and y-dimension so that each row in tensor **438** has 2 blocks. Each column in tensor **438** also has 2 blocks.
[0090] The block and partitioned structures can be applied to both input activation tensor **220** and weight tensor **222**. Each of the input activation tensor **220** and weight tensor **222** may be blocked and partitioned in a similar manner but in different dimensions so that the pairing of input activation tensor **220** and weight tensor **222** can predictably limit the number of computations. FIG. **4**B illustrates several examples of such pairing of tensors. In operation **450**, partitioned-x tensor **406** may represent the weight tensor **222** and partitioned-y tensor **422** may represent the input activation tensor **220**. The tensor **406** and tensor **422** both have a partitioned structure but the former has the partitions in a first dimension and the latter has the partitions in a second dimension different from the first dimension. Rows of tensor **406** and columns of tensor **422** have a fixed number of elements. Hence, operation **450** can have a maximum of 4 multiply-accumulate operations per dot-product.
[0091] In operation **460**, block-x and partitioned-x tensor **408** may represent the weight tensor **222** and block-y and partitioned-y tensor **416** may represent the input activation tensor **220**. The tensor **408** and tensor **416** both have block structure and partitioned structure, but both blocks are partitions in different dimensions. In this case, rows of tensor **408** and columns of tensor **416** have a fixed number of blocks. Hence, operation **460** can have the maximum of 1 single instruction multiple data (SIMD) block multiply-accumulate operations per dot-product.
[0092] In operation **470**, block-x and partitioned-xy tensor **428** may represent the weight tensor **222** and block-y and partitioned-xy tensor **436** may represent the input activation tensor **220**. The tensor **428** and tensor **436** both have block structure and partitioned structure, but the blocks are divided in different dimensions. In this case, both rows and columns of tensor **428** and the row and columns of tensor **436** have a fixed number of blocks. Hence, operation **470** can have the maximum of 1 single instruction multiple data (SIMD) block multiply-accumulate operations per dot-product.
[0093] FIG. **5** is a flowchart depicting an example process for performing operations related to a sparse neural network, according to an embodiment. The process may be performed by a computing device, such as computing device **100**. The computing device may be equipped with an AI accelerator **300** and may perform one or more steps of this process using the AI accelerator **300**. However, in some embodiments, the process may also be performed using a CPU. The process may be embodied as software algorithm that may be stored as computer instructions that are executable by a processor. The instructions, when executed by the processor, cause the processor to perform various steps described in FIG. **5**.
[0094] The computing device initiates **510** a neural network with a plurality of nodes. The structure of the neural network may depend on its type, which can be CNN, RNN, LSTM, etc. The structures and operations of the nodes can be different among the nodes. For one or more nodes, each may be associated with a weight tensor and an activation tensor. The structure and operation related to the tensors are discussed in FIGS. **2**A and **2**B. The neural network initiated may be saved in system memory **108** or storage unit **110**. The weight tensor in each node may include a plurality of data values. The initial data values may be initiated randomly or based on expected values. In some cases, the initial data values in the weight tensor may be initiated with a number of zeros.

[0095] The computing device imposes **520** one or more structural constraints to limit the distribution of active values of the weight tensor. The constraints may be based on one or more code instructions in training the neural network that defines the configuration of the neural network. The structure can be blocky or partitioned or both. The blocks and partitions may also be assigned in any dimensions, as discussed in FIG. **4**A. A structural constraint may be node specific or be applied across a set of nodes. For example, each node can have the same constraints, different constraints, or no constraints. The structural constraint restricts the distribution of active values in the weight tensor and also the number of active values in the tensor. Hence, if a node is imposed with a structural constraint, the node is also a sparse node that has a small number of active values.
[0096] For a node, a structural constraint may also be imposed for activation tensors by way of the K-winner activation function. For the nodes in the input layer **202**, the input activation tensor **220** may likely be a dense tensor because the input data is often data such as image data, speech data, etc. As such, a node in the input layer may be a sparse-weight, dense-activation node, or simply sparse-dense node. After weight tensor **222** and a K-winner activation function is applied, the output activation tensor **228** can be sparse. For example, the K-winner activation function can limit the number of active values in the output activation tensor **228** and force the loser data values to zeros. The output activation tensor **228** becomes in the input activation tensor **220** of the next node. The next node can be a sparse-sparse node. The K-winner activation function may take the form of training the neural network that defines the configuration of the neural network.
[0097] While K-winner activation function is described as an example of a sparse activation function, other sparse activation functions may also be used in various embodiments. A sparse activation function is an activation function that results in a sparse output the activation function is applied to the computation result in a neural network node. For example, in K-winner activation function, the number of active values in the output may be limited by K. Alternatively, or additionally, a threshold approach may be used as a sparse activation function. Values that are below (or above) the threshold are set to inactive (e.g., set to zeros). The threshold may be global or local, static or dynamic. The threshold is applied to an entire tensor in the global approach while the threshold is only applied to a certain subset of data (e.g., a block or a partition) in a local approach. In a static approach, a predetermined threshold value may be used. In a dynamic approach, a threshold value may vary based on factors to be determined during the training. For example, statistics may be performed on a set of values on the fly to determine a dynamic threshold cutoff to set some of the values to zeros.
[0098] The structure constraint, similar to the weight tensor, can be blocked or partitioned. In blocky K-winner, for a given output activation tensor, a processor of the computing device may divide the tensor into blocks. For each block, the processor may perform statistics on an aggregation of the blocks, such as by taking the maximum, the minimum, the sum of the values in each block. The top K winning blocks will be selected as the dense blocks. The values in the rest of the blocks will be forced to zero. In partitioned K-winner, for a given output activation tensor, the processor may divide the output activation tensor into partitions. For each partition, the processor may select a fixed number of highest values in the partition as the winners. In this context, if K-winner is applied to an entire tensor, the K-winner approach may be referred to as a global K-winner. If K-winner is applied to a subset of the tensor, such as a dimension, a block, or a partition of the data, the K-winner approach may be referred to as local K-winner.
[0099] The computing device may train **530** the neural network using the structural constraints. The computing device may use one or more processors, such as an AI accelerator **300**, a CPU, or in combination, to perform different computations associated with training of the neural network. The training **530** may include forward propagation **540** and backpropagation **550**. In forward propagation **540**, the processor performs computations as defined by each node in the forward direction as illustrated in FIG. **2**A. In one or more nodes, the computation may include a linear operation between an input activation tensor **220** and a weight tensor **222** followed by a non-linear

operation, as illustrated in FIG. **2**B. In backpropagation **550**, the processors may adjust the weight values in the weight tensors using techniques such as coordinate descent and also based on the structural constraints imposed on one or more nodes.

[0100] In forward propagation **540**, different operations may be performed based on the sparsity of a node. The operations may include pre-processing, multiply-accumulation, and post-processing of tensors. According to an embodiment, in a sparse-dense node, a processor may, as part of a pre-processing, transpose **542** a sparse weight tensor based on the structure of the weight tensor. In one embodiment, the transpose may align the partitions in a structured tensor with the lanes in a computation core **320** that are used to transmit data to multiply circuits **330**. For example, a computation core **320** of a processor may include 64 multiply circuits **330** and N lanes for transmitting values to the multiply circuits **330**. The weight tensor may be divided into N partitions and be transposed to align the partitions with the lanes. The partition of the weight tensor and hardware structure of an example computation core **320** according to an embodiment will be further illustrated in FIG. **6**.

[0101] Alternatively, or additionally, the processor may compress **544** to convert the sparse weight tensor into a dense weight tensor by removing zeros in the sparse weight tensor as part of the pre-processing. The processor may record the location addresses of the active values or the dense blocks (in the case of block structure). Since the sparse weight tensor is structured, the identification of the active value locations is significantly reduced. If the weight tensor is partitioned, the numbers of active values or dense blocks may be the same or similar in each partition. As such, the computation with respect to each partition is expected to be completed with the same number of cycles or a similar number of cycles, thereby improving the throughput and efficiency of the processor.

[0102] The processor may perform **546** multiply-accumulate operations between the compressed weight tensor and input activation tensor. The multiply-accumulate operations may be performed in parallel by a number of multiply circuits **330** in parallel. The computation results of the multiply circuits **330** are aggregated in an adder tree **360**. The processor may also apply 548 an activation function to the output of the adder tree **360**. The activation function may be a dense activation function such as ReLU or tanh. The activation function may also be a sparse activation function such as K-winner. The activation function may further be a sparse and structured activation function such as blocky K-winner or partitioned K-winner. After completing the computations of a node, the processor may perform computations on a subsequent node in the forward direction of the neural network until an inference result is made. The inference result is compared to the actual label of a training sample.

[0103] In backpropagation **550**, the processor may adjust **552** the weight values in weight tensors of the node under the structural constraints. For example, the weight values may be adjusted using techniques such as coordinate descent to change the values in directions that will more likely for the neural network to generate the correct inference result. In adjusting the weight values, only certain locations of the values are allowed to be non-zero and the active values are distributed according to the structural constraints.

[0104] After the neural network is trained with training samples, the neural network may be used to make inferences of actual samples. The inference may be performed using the forward propagation **550**. The inference is also accelerated because the trained weight tensors are structured. The process used may also have hardware architecture that is efficient in processing those structured weight tensors.

Example Hardware Architecture for Sparse Weight Tensor

[0105] FIG. **6** is a block diagram illustrating an example circuitry of a computation core **320**, according to an embodiment. In FIG. **6** and other examples in this disclosure, the specific numbers are only provided as examples only.

[0106] Computation core **320** is a circuit that is efficient at processing data in a sparse-weight,

dense-activation node. The sparsity processing circuit **354** receives a structured weight tensor **610** that has 25 rows and 64 columns. The weight tensor **610** is a block partitioned tensor. The shaded areas **620** represent the dense blocks in the weight tensor **610**, which has a density of 12%. Each shaded area **620** is a 1×8 block that has 8 active values. The weight tensor **610** is partitioned vertically with 8 partitions **630**. Each partition **630** is set to have three dense blocks **620**.

[0107] Sparsity processing circuit **354** may include multiple lanes (e.g., **8** lanes) for transmitting data to multiply circuits **330**. Each lane may be a set of multi-ported memory. The memory may take the form of suitable static random access memory (SRAM), memristors, and registers. For example, in a FPGA or ASIC implementation, SRAM may be used. In general, the various forms of memory used in an AI processor may simply be referred to as RAM. Each lane may have multiple rows of fixed size (e.g., 8 byte) and may process data in a first-in-first-out manner. In one example, each data value may be a byte, and since a block **620** has 8 values, each lane is 8 bytes in width.

[0108] The number of lanes may correspond to the number of partitions **630** in weight tensor **610**. The weight tensor **610** may be stored in a memory such as internal memory **310** or system memory **106** shown in FIG. **3** in a different configuration where the partitions may not align the lanes of sparsity processing circuit **354**. After sparsity processing circuit **354** receives the weight tensor **610**, sparsity processing circuit **354** may perform transpose operation to align the partitions **630** and the lanes. In turn, the weight tensor **610** may be compressed to a dense tensor with 3 rows only by removing the sparse blocks. The values in the dense tensor may be transmitted to the 64 multiply circuits, for example, in three operating cycles. Since each lane in sparsity processing circuit **354** has a balanced number of active values, the computation may be completed in three operating cycles. If weight tensor **610** is unstructured, in the worst case the computation core **320** may need to take 24 cycles to complete the operation.

[0109] The operation of sparsity processing circuit **354** may be further illustrated using an example of CNN with actual numbers. While specific numbers are provided in this example and other examples in this disclosure, the numbers are only for illustration and embodiments are not limited to those specific numbers. In one embodiment, a convolution node in the CNN may be associated with a weight tensor that has the dimension of 64 (w)×64 (z)×5 (y)×5 (x), an input activation tensor that has the dimension of 64 (z)×14 (y)×14 (x). In other words, the weight tensor represents 64 different kernels of the size 64 (z)×5 (y)×5 (x) in the w dimension that are being convolved with an input data of the dimension of 64 (z)×14 (y)×14 (x). The convolution generates a result of 64 (z)×10 (y)×10 (x).

[0110] Sparsity processing circuit **354** may transpose the weight tensor and the activation tensor so that the Z (channel) loop is the innermost. As such the weight tensor and the activation tensor are respectively transposed to 64 (w)×5 (z)×5 (y)×64 (x) and 14 (z)×14 (y)×64 (x). The result generated is in the dimension of 10 (z)×10 (y)×64 (x), but can be transposed back to the original dimension. The transpose may be performed by fetching the data values from internal memory **310** in a different order than how the data values are stored in the memory **310**. The 64 (w)×5 (z)×5 (y)×64 (x) tensor may be supported by, for example, SIMD 64 hardware. For SIMD 64 hardware, RAM for weight tensors could be individually addressed in groups of 8 bytes, while RAM for activation tensors could have smaller individually RAM sizes for more flexible addressing. For a block size of 64, there can be 25 blocks in the 5×5 kernel. For a block size of 32, there can be 50 blocks in the 5×5 kernel. For a block size of 16, there can be 100 blocks in the 5×5 kernel. For block size of 64, the 64 (w)×5 (z)×5 (y)×64 (x) may be aligned with the eight 8-byte lanes in FIG. **6**. The weight tensor may be structured and may potentially yield 100% efficiency in the SIMD 64 unit.

[0111] With the smaller block sizes, it is possible that the blocks might not be evenly distributed in the weight tensor (unbalanced), leading to less than 100% computation efficiency, resulting in more cycles for a convolution. To address this, sparsity processing circuit **354** may include multiplexors to distribute the activation values across lanes, plus additional instances of weight blocks. For

example, for a block size of 16, there will be 4 lane groups.
[0112] With respect to parallelism, RAM may be used to store weight and activation tensor values. With 32 bits output for each RAM, there is room for 1024 blocks. Weight memories may be only 31% full. Activation memories may be only 19% full. The RAM used may be of 72 bit size, 36 bit size, or 18 bit size. Since RAM of 72 bit size (8 bytes with a parity for each byte) can output 64 bits but could be eight times of the size of a RAM of 36 bit size, the fill factor is four time worse for RAM of 72 bit size. Extra space could be used to store weights for other convolution layers. Since the activation is stored in original size, only 14×14=196 cycles are needed to load in a new activation. The architecture can be replicated multiple times for parallel operation. For example, 10× replications will result in 3200 cycles, plus 196 cycles to load in new activation. Sparsity processing circuit **354** may include 80 RAM of 72 bit size (weights) and 160 RAM of 36 bit size (activation), or any suitable ratios of two sets of RAMs.
[0113] With a block size of 16, there are 5×5×4=100 blocks in each 5×5×64 weight volume. Assuming a non-zero block density of 0.1, 10 out of the 100 blocks will be non-zero. With even distribution, 3 non-zero blocks will go to two of the four lane groups, and 2 non-zero blocks will go to the other two (3+3+2+2). This results in 3 cycles to complete a convolution, with an efficiency of 83%. In the general case, all 10 of the blocks might be in one of the lane groups (10+0+0+0), which would need more weight RAM utilization in that lane, plus cycles to complete a convolution, with an efficiency of 25%. However, if multiplexers (512 1-of-4 instances) are added to enable a cross-lane distribution of the activation, efficiency can be increased to 83% by rebalancing RAM utilization. FIG. **7** illustrates the hardware implementation for this example with multiplexers **710**.
Example Sparse-Sparse Node Implementation
[0114] FIG. **8** is a flowchart depicting an example process for performing operations related to a sparse-sparse node in a neural network, according to an embodiment. The process may be performed by a computing device, such as computing device **100**. The computing device may be equipped with an AI accelerator **300** and may perform one or more steps of this process using the AI accelerator **300**. However, in some embodiments, the process may also be performed using a CPU. The process may be embodied as software algorithm that may be stored as computer instructions that are executable by a processor. The instructions, when executed by the processor, cause the processor to perform various steps described in FIG. **8**. The process illustrated in FIG. **8** may be an example of subprocess of the process discussed in FIG. **5**.
[0115] A processor performs the operation related to a sparse-sparse node that is associated with a sparse weight tensor and a sparse input activation tensor. The processor generates **810** a first bit vector for the sparse weight tensor and a second bit vector for the input activation tensor. A bit vector is a string of bits that encode the locations of active values in a tensor. A bit vector may have a first value (e.g., 1) that represents the dense byte value in a tensor and a second value (e.g., 0) that represents the inactive value in the tensor. For example, if a tensor has 1600 bytes, each byte representing a value, the bit vector will be a string of 1600 bits with 1s corresponding to the locations where active values are identified.
[0116] The processor performs **820** a bitwise AND operation between the first bit vector and the second bit vector to generate a sparse operand bit vector. The sparse operand bit vector is the product of the first and second bit vectors. In a linear operation that can be computed using a dot product, only the active values generated in the output will be at the locations where the weight tensor and the activation tensor both have active values. The bitwise AND operation only keeps the value 1 in those locations and will turn locations that have a zero in either bit vector into zero. Hence, the sparse operand bit vector represents the locations where both the weight tensor and the activation tensor have active values.
[0117] The processor generates **830** weight addresses and activation addresses with active values based on the sparse operand bit vector. Each weight address and activation address are memory

addresses of the data values that are stored in a memory location. The memory may be the system memory **106**, internal memory **310**, activation data buffer **352** or weight buffer **350**. Each weight address has a corresponding activation address whose data is in the same corresponding position in the weight tensor and the activation tensor.

[0118] The weight tensor and the activation tensor can be compressed or uncompressed in this process. For example, if data values are directly fetched from system memory **106** or internal memory **310**, the tensors may be uncompressed. The tensors that have been compressed (e.g., removing zeros) by sparsity processing circuit **354** may also be used. If the weight tensor and activation tensor are stored in memory uncompressed, the 1-bit positions in the sparse operand bit vector correspond to the active value addresses for the weight tensor and activation tensor pairs. If the weight tensor and activation tensor are compacted to remove zero values, the counts of the 1-bits in the weight and activation bit vectors, up to but not including the location of the 1-bit positions in the sparse operand bit vector correspond to the weight tensor's and activation tensor's active value address. The generation of weights and activation value addresses in their respective compacted memories from the bit vectors may be performed by one or more gating networks.

[0119] The processor fetches **840** the active values from the weight tensor and the activation tensor based on the weight addresses and activation addresses. The processor transmits **850** the fetched active values to multiply circuits **330** to perform multiply-accumulate operations. Since only data corresponding to locations where both tensors have active values are transmitted to multiply circuits **330**, the number of multiply-accumulate operations is significantly reduced.

[0120] FIG. **9**A is a block diagram illustrating the generation of a sparse operand bit vector, according to an embodiment. FIG. **9**B is a conceptual diagram illustrating the generation of a sparse operand bit vector. FIG. **9**A shows that the sparse operand bit vector is generated by a bitwise AND operation between a weight bit vector and an activation bit vector. The AND operation may simply be achieved in hardware by passing the bits in the two bit vectors through an AND gate. FIG. **9**B shows an example of a weight bit vector and an example of an activation bit vector. Position values from the weight bit vector are illustrated in solid-lined boxes and position values from the activation bit vector are illustrated in dash-lined boxes. In the weight bit vector, only positions 3, 9 and 12 are dense. In the activation bit vector, only positions 1, 4, 6, 9, 11, 14, and 15 are dense. By using the bitwise AND operation, the sparse operand bit vector has only one value of 1 at the 9th position.

[0121] Upon reading this disclosure, those of skill in the art will appreciate still additional alternative designs for processing nodes. Thus, while particular embodiments and applications have been illustrated and described, it is to be understood that the invention is not limited to the precise construction and components disclosed herein and that various modifications, changes and variations which will be apparent to those skilled in the art may be made in the arrangement, operation and details of the method and apparatus disclosed herein without departing from the spirit and scope of the present disclosure.

Claims

1. A neural processor circuit, comprising: a memory circuit configured to store a sparse weight tensor and a sparse activation tensor corresponding to a node of a neural network; a sparsity processing circuit coupled to the memory circuit, the sparsity processing circuit configured to: determine one or more activation values of the sparse activation tensor, and one or more weight values of the sparse weight tensor that would yield non-zero result values responsive to being applied with mathematical operations, and fetch the determined one or more activation values and the one or more weight values; and an operation circuit coupled to the sparsity processing circuit, and configured to: receive the fetched one or more activation values and the fetched one or more active weight values, and perform the mathematical operations between the fetched one or more

active activation values and the fetched one or more active weight values.

2. The neural processor circuit of claim 1, wherein the sparsity processing circuit is further configured to generate a first bit vector from the sparse weight tensor and a second bit vector from the sparse activation tensor, the first bit vector indicating locations of the one or more active weight values in the sparse weight tensor, and the second bit vector indicating one or more active activation values in the sparse activation tensor.

3. The neural processor circuit of claim 2, wherein the sparsity processing circuit is further configured to determine locations of the one or more activation values and the one or more weight values by performing a bitwise AND operation between the first bit vector and the second bit vector.

4. The neural processor circuit of claim 1, wherein the sparse weight tensor has a defined distribution of active weight values and inactive weight values.

5. The neural processor circuit of claim 1, wherein the mathematical operations comprise multiply operations between the one or more active values and the one or more weight values.

6. The neural processor circuit of claim 5, wherein the operation circuit comprises: a plurality of multiply circuits configured to perform multiply operations, and an adder tree configured to perform accumulate operations on results of the multiply operations to generate accumulated values.

7. The neural processor circuit of claim 6, wherein the operation circuit is further configured to: generate an intermediate tensor with intermediate values derived from the accumulated values; and maintain a subset of highest intermediate values but zero out remaining intermediate values to generate an output tensor of the node.

8. The neural processor circuit of claim 1, wherein the sparsity processing circuit comprises lanes configured to send the determined one or more weight values to the operation circuit.

9. The neural processor circuit of claim 8, wherein each of the lanes is assigned to weight values in a block or a partition in the sparse weight tensor.

10. The neural processor circuit of claim 1, wherein the sparsity processing circuit is configured to compress at least one of the sparse weight tensor or the sparse activation tensor into a dense weight tensor or the dense activation tensor.

11. A method of processing using a neural network, comprising: storing a sparse weight tensor and a sparse activation tensor corresponding to a node of the neural network in a memory circuit; determining, by a sparsity processing circuit, one or more activation values of the sparse activation tensor, and one or more weight values of the sparse weight tensor that would yield non-zero result values responsive to being applied with mathematical operations; fetching the determined one or more activation values and the one or more weight values by the sparsity processing circuit; receiving, by an operation circuit, the fetched one or more activation values and the fetched one or more active weight values; and performing the mathematical operations between the fetched one or more active activation values and the fetched one or more active weight values.

12. The method of claim 11, further comprising, generating a first bit vector from the sparse weight tensor and a second bit vector from the sparse activation tensor by the sparsity processing circuit, the first bit vector indicating locations of the one or more active weight values in the sparse weight tensor, the second bit vector indicating one or more active activation values in the sparse activation tensor.

13. The method of claim 12, further comprising determining, by the sparsity processing circuit, locations of the one or more activation values and the one or more weight values by performing a bitwise AND operation between the first bit vector and the second bit vector.

14. The method of claim 11, wherein the sparse weight tensor has a defined distribution of active weight values and inactive weight values.

15. The method of claim 11, wherein the mathematical operations comprise multiply operations between the one or more active values and the one or more weight values.

16. The method of claim 15, further comprising: performing multiply operations by the operation circuit; and performing, by the operation circuit, accumulate operations on results of the multiply operations to generate accumulated values.

17. The method of claim 16, further comprising: generating an intermediate tensor with intermediate values derived from the accumulated values; and maintaining a subset of highest intermediate values but zeroing out remaining intermediate values to generate an output tensor of the node.

18. The method of claim 11, further comprising sending the determined one or more weight values to the operation circuit via lanes of the sparsity processing circuit.

19. The method of claim 18, wherein each of the lanes is assigned to weight values in a block or a partition in the sparse weight tensor.

20. The method of claim 11, wherein compressing at least one of the sparse weight tensor or the sparse activation tensor into a dense weight tensor or the dense activation tensor by the sparsity processing circuit.